

SQL ASSIGNMENT-1

Part 1: Simple Table Creation

Exercise 1: Student Information Table

Create a table called students to store student information.

What you need to store:

- Student ID (whole numbers, automatically increases)
- Student name (different length names like "Ali" or "Muhammad Abdullah")
- Roll number (always 6 characters like "CS0001")
- Email address (different lengths)
- Age (whole numbers only)
- CGPA (numbers like 3.75, exact values needed)
- Is scholarship student? (Yes/No)
- Date of birth (dates only)
- Phone number (always 11 digits like "03001234567")

Your Task: Write the CREATE TABLE command with appropriate data types

The screenshot shows a SQL IDE with a query editor and a results/output pane. The query editor contains the following SQL code:

```
1 • CREATE DATABASE assignment_db;
2 • USE assignment_db;
3 • # SHOW TABLES;
4
5 -- ***EXERCISE 1***
6 • CREATE TABLE students(
7     ID INT PRIMARY KEY auto_increment NOT NULL,
8     Stu_Name VARCHAR(255),
9     Roll_no CHAR(6),
10    Email VARCHAR(255),
11    Age INT,
12    CGPA DECIMAL(3,2),
13    IsScholar BOOLEAN,
14    DOB DATE,
15    Phno CHAR(11)
16 );
17
18 • SELECT * FROM students;
```

The results pane shows a table with 9 columns: ID, Stu_Name, Roll_no, Email, Age, CGPA, IsScholar, DOB, and Phno. All cells are empty. Below the table, the 'Action Output' pane shows the following messages:

#	Time	Action	Message
2	18:41:07	CREATE TABLE students(ID INT PRIMARY KEY auto_increment NOT NULL, Stu_Name VARCHAR(255), Roll_no CHAR(6), Email VARCHAR(255), Age INT, CGPA DECIMAL(3,2), IsScholar BOOLEAN, DOB DATE, Phno CHAR(11));	Error Code: 1050. Table 'students' already exists
3	18:41:21	USE assignment_db	0 row(s) affected
4	18:41:21	SELECT * FROM students LIMIT 0, 1000	0 row(s) returned

Exercise 2: Library Books Table

Create a table called books for a library system.

What you need to store:

- Book ID (whole numbers, starts from 1)
- Book title (variable length)
- Author name (variable length up to 80 characters)
- ISBN (always 13 characters like "9781234567890")
- Category (must be one of: Fiction, Non-Fiction, Science, History)
- Price (exact money values like 599.99)
- Number of pages (whole numbers)
- Is available for lending? (Yes/No)
- Publication date (dates only)
- Brief description (long text)

Your Task: Write the CREATE TABLE command

Code:

```
CREATE TABLE books (  
  book_id INT AUTO_INCREMENT PRIMARY KEY,  
  title VARCHAR(150),  
  author VARCHAR(80),  
  isbn CHAR(13),  
  category ENUM('Fiction', 'Non-Fiction', 'Science', 'History'),  
  price DECIMAL(10,2),  
  pages INT,  
  is_available BOOLEAN,  
  publication_date DATE,  
  brief_description TEXT  
);
```

Output				
Action Output				
#	Time	Action	Message	Duration / Fetch
90	17:25:00	USE ass	0 row(s) affected	0.000 sec
91	17:33:30	CREATE TABLE students(student_ID INT AUTO_INCREMENT PRIMARY KEY, student_name...	0 row(s) affected	0.031 sec
92	17:43:47	CREATE TABLE books (book_id INT AUTO_INCREMENT PRIMARY KEY, title VARCHAR...	0 row(s) affected	0.016 sec

Part 2: Understanding Different Number Types

Exercise 3: Online Store Orders

Create a table called orders that handles different types of numbers.

What you need to store:

- Order ID (very large numbers for millions of orders)
- Customer ID (regular whole numbers)
- Order amount (exact money - no rounding errors!)
- Tax amount (exact money)
- Shipping weight (approximate decimals like 2.567 kg)
- Number of items (small whole numbers 1-50)
- Is express delivery? (Yes/No)
- Order date and time together
- Discount percentage (approximate decimals)

Your Task:

1. Create the table with proper numeric data types
2. Insert these 3 sample orders:

Order 1: Customer 101, Amount 1250.75, Tax 125.08, Weight 3.2kg, 5 items, Express delivery, Today's date

Order 2: Customer 205, Amount 89.99, Tax 9.00, Weight 0.5kg, 1 item, Regular delivery, Yesterday

Order 3: Customer 156, Amount 2999.50, Tax 299.95, Weight 8.7kg, 12 items, Express delivery, Today

Questions to Answer:

1. Why did you choose DECIMAL for money instead of FLOAT?
2. Why use BIGINT for Order ID?
3. What happens if you try to insert a negative number of items?

Code:

```
CREATE TABLE orders (  
  order_id BIGINT AUTO_INCREMENT PRIMARY KEY,  
  customer_id INT,  
  order_amount DECIMAL(10,2),  
  tax_amount DECIMAL(10,2),  
  shipping_weight FLOAT,  
  number_of_items TINYINT,  
  is_express BOOLEAN,  
  order_datetime DATETIME,  
  discount_percentage FLOAT  
);  
  
INSERT INTO orders  
(customer_id, order_amount, tax_amount, shipping_weight, number_of_items, is_express,  
order_datetime, discount_percentage)  
VALUES  
(101, 1250.75, 125.08, 3.2, 5, TRUE, NOW(), 5.5),  
(205, 89.99, 9.00, 0.5, 1, FALSE, NOW() - INTERVAL 1 DAY, 2.0),  
(156, 2999.50, 299.95, 8.7, 12, TRUE, NOW(), 10.0);  
  
SELECT * FROM orders
```

The screenshot shows a database management tool interface. At the top, there's a 'Result Grid' with columns: order_id, customer_id, order_amount, tax_amount, shipping_weight, number_of_items, is_express, order_datetime, and discount_percentage. The grid contains three rows of data. Below the grid, there's a tab labeled 'orders 1 x'. Underneath, there's an 'Output' section with a dropdown menu set to 'Action Output'. This section displays a log of database actions with columns: #, Time, Action, Message, and Duration / Fetch. The log shows five actions, including the creation of tables and the insertion of data into the 'orders' table.

#	Time	Action	Message	Duration / Fetch
91	17:33:38	CREATE TABLE students(student_ID INT AUTO_INCREMENT PRIMARY KEY, student_name...	0 row(s) affected	0.031 sec
92	17:43:47	CREATE TABLE books(book_id INT AUTO_INCREMENT PRIMARY KEY, title VARCHA...	0 row(s) affected	0.016 sec
93	17:55:06	CREATE TABLE orders(order_id BIGINT AUTO_INCREMENT PRIMARY KEY, customer...	0 row(s) affected	0.015 sec
94	17:58:45	INSERT INTO orders (customer_id, order_amount, tax_amount, shipping_weight, number_of_te...	3 row(s) affected Records: 3 Duplicates: 0 Warnings: 0	0.016 sec
95	17:59:54	SELECT * FROM orders LIMIT 0, 1000	3 row(s) returned	0.000 sec / 0.000 sec

Questions to Answer:

1. Why did you choose DECIMAL for money instead of FLOAT?

#IT'S CAUSE Decimal gives precise answers compared to float. Float can cause rounding errors which is not acceptable in financial data.

2. Why use BIGINT for Order ID?

BigInt stores large numbers since there can be millions of order while int has a storage size of 4 bytes compared to 8 bytes for bigint

3. What happens if you try to insert a negative number of items?

```
INSERT INTO orders (Cust_ID,Order_Amt,Tax_Amt,Shipping_Wt,no_Items,IsExpress,
OrderDate, Discount_percent) VALUES (156, 2999.50, 299.95, 8.7, -12, 1, NOW(), 0.0);
# error generated constraint is violated
```

Part 3: Text and String Types

Exercise 4: User Registration System

Create a users table that shows the difference between CHAR and VARCHAR.

What you need to store:

- User ID (whole numbers)
- Username (3 to 25 characters)
- Country code (always 2 letters like "PK", "US")
- State code (always 3 letters like "SND", "PUN")
- Full name (variable length up to 100 characters)
- Password (always 32 characters - encrypted)
- Account type (Premium, Basic, Guest)
- About me (long biographical text)
- Verification code (always 6 digits like "123456")

Your Task:

1. Create the table using CHAR for fixed-length fields and VARCHAR for variable-length fields
2. Insert 5 different users with varying name lengths

Code:

```
CREATE TABLE users (
  user_id INT AUTO_INCREMENT PRIMARY KEY,
  username VARCHAR(25),
  country_code CHAR(2),
  state_code CHAR(3),
  full_name VARCHAR(100),
  password CHAR(32),
  account_type ENUM('Premium', 'Basic', 'Guest'),
```

```

about TEXT,
verification_code CHAR(6)
);
INSERT INTO users
(username, country_code, state_code, full_name, password, account_type, about,
verification_code)
VALUES
('Ali', 'PK', 'PUN', 'Ali Khan', '12345678901234567890123456789012', 'Basic', 'Student',
'123456'),
('Sara123', 'PK', 'SND', 'Sara Ahmed', '12345678901234567890123456789012', 'Premium',
'Designer', '234567'),
('JohnDoe', 'US', 'CAL', 'John Doe', '12345678901234567890123456789012', 'Guest',
'Visitor',
'345678'),
('AyeshaK', 'PK', 'KPK', 'Ayesha Khan', '12345678901234567890123456789012', 'Basic',
'Teacher', '456789'),
('MuhammadAbdullah', 'PK', 'PUN', 'Muhammad Abdullah',
'12345678901234567890123456789012', 'Premium', 'Engineer', '567890');
SELECT * FROM users;

```

The screenshot shows a database management interface. At the top, a 'Result Grid' displays the contents of the 'users' table. Below it, the 'Output' pane shows a log of SQL operations executed.

user_id	username	country_code	state_code	full_name	password	account_type	about	verification_code
1	Ali	PK	PUN	Ali Khan	12345678901234567890123456789012	Basic	Student	123456
2	Sara123	PK	SND	Sara Ahmed	12345678901234567890123456789012	Premium	Designer	234567
3	JohnDoe	US	CAL	John Doe	12345678901234567890123456789012	Guest	Visitor	345678
4	AyeshaK	PK	KPK	Ayesha Khan	12345678901234567890123456789012	Basic	Teacher	456789
5	MuhammadAbdullah	PK	PUN	Muhammad Abdullah	12345678901234567890123456789012	Premium	Engineer	567890

#	Time	Action	Message	Duration / Fetch
99	18:24:26	CREATE TABLE users (user_id INT AUTO_INCREMENT PRIMARY KEY, username VAR...	0 row(s) affected	0.047 sec
100	18:27:16	INSERT INTO users (username, country_code, state_code, full_name, password, account_typ...	5 row(s) affected Records: 5 Duplicates: 0 Warnings: 0	0.000 sec
101	18:27:35	SELECT * FROM users LIMIT 0, 1000	5 row(s) returned	0.000 sec / 0.000 sec

Part 4: Dates and Times

Exercise 5: Event Management

Create an events table for managing school events.

What you need to store:

- Event ID (whole numbers)

- Event name (variable text)
- Event date (date only)
- Start time (time only)
- End time (time only)
- Registration deadline (date and time together)
- Created timestamp (automatically set when record is added)
- Is public event? (Yes/No)
- Max participants (small whole numbers)

Your Task:

1. Create the table
2. Insert 3 upcoming events with different dates and times
3. Write a query to find events happening today

Think About These Questions

1. When would you use CHAR instead of VARCHAR?
2. Why is DECIMAL better than FLOAT for storing prices?
3. What's the difference between DATE and DATETIME?
4. When should you use ENUM data type?
5. What happens if you try to store text in an INT column

Code:

```
CREATE TABLE events (
  id INT AUTO_INCREMENT PRIMARY KEY,
  event_name VARCHAR(100) NOT NULL,
  event_date DATE NOT NULL,
  start_time TIME,
  end_time TIME,
  reg_deadline DATETIME,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  is_public BOOLEAN DEFAULT 1,
  max_participants SMALLINT
);

INSERT INTO events (event_name, event_date, start_time, end_time, reg_deadline,
is_public,
max_participants)
```

VALUES

```
('Fair', '2026-03-31', '09:00:00', '15:00:00', '2026-03-30 17:00:00', 1, 200),  
('Debate', '2026-04-05', '11:00:00', '13:30:00', '2026-04-02 12:00:00', 1, 50),  
('Dance', '2026-03-31', '14:00:00', '15:00:00', '2026-03-31 10:00:00', 0, 30);  
*/  
/*
```

```
SELECT * FROM events  
WHERE event_date = CURDATE();  
*/
```

#Q1 When would you use CHAR instead of VARCHAR?

when i need fixed length of characters

#Q2: Why is DECIMAL better than FLOAT for storing prices

it's cause decimal stores the values with greater precision without rounding, while float has less precision with rounding errors

#Q3: What's the difference between DATE and DATETIME?

DATE STORES ONLY THE DATE (YYYY-MM-DD) BUT DATETIME CONSISTS OF TIME

PART TOO (YYYY-MM-DD HH:MM:SS)

#Q4: When should you use ENUM data type?

WHEN I NEED TO STORE CERTAIN FIXED NUMBER OF CATEGORIES

#Q5: What happens if you try to store text in an INT column?

IT WILL GIVE A DATA TYPE MISMATCH ERROR AND REJECT THE QUERY

The screenshot shows a database management interface. At the top, there's a 'Result Grid' with a toolbar for filtering, editing, and exporting. Below it is a table with 10 columns: id, event_name, event_date, start_time, end_time, reg_deadline, created_at, is_public, and max_participants. The table contains three rows of data for events named 'Fair', 'Debate', and 'Dance'. Below the table, there's a tab labeled 'events 4'. At the bottom, there's an 'Output' section with a dropdown menu set to 'Action Output'. It displays a log of three SQL queries executed at 18:56:25, 18:56:38, and 18:56:38, along with their results: 0 rows returned, 0 rows affected, and 3 rows returned respectively.

id	event_name	event_date	start_time	end_time	reg_deadline	created_at	is_public	max_participants
1	Fair	2026-03-31	09:00:00	15:00:00	2026-03-30 17:00:00	2026-03-31 10:25:04	1	200
2	Debate	2026-04-05	11:00:00	13:30:00	2026-04-02 12:00:00	2026-03-31 10:25:04	1	50
3	Dance	2026-03-31	14:00:00	15:00:00	2026-03-31 10:00:00	2026-03-31 10:25:04	0	30

#	Time	Action	Message
10	18:56:25	SELECT * FROM events WHERE event_date = CURDATE() LIMIT 0, 1000	0 row(s) returned
11	18:56:38	USE assignment_db	0 row(s) affected
12	18:56:38	SELECT * FROM events LIMIT 0, 1000	3 row(s) returned